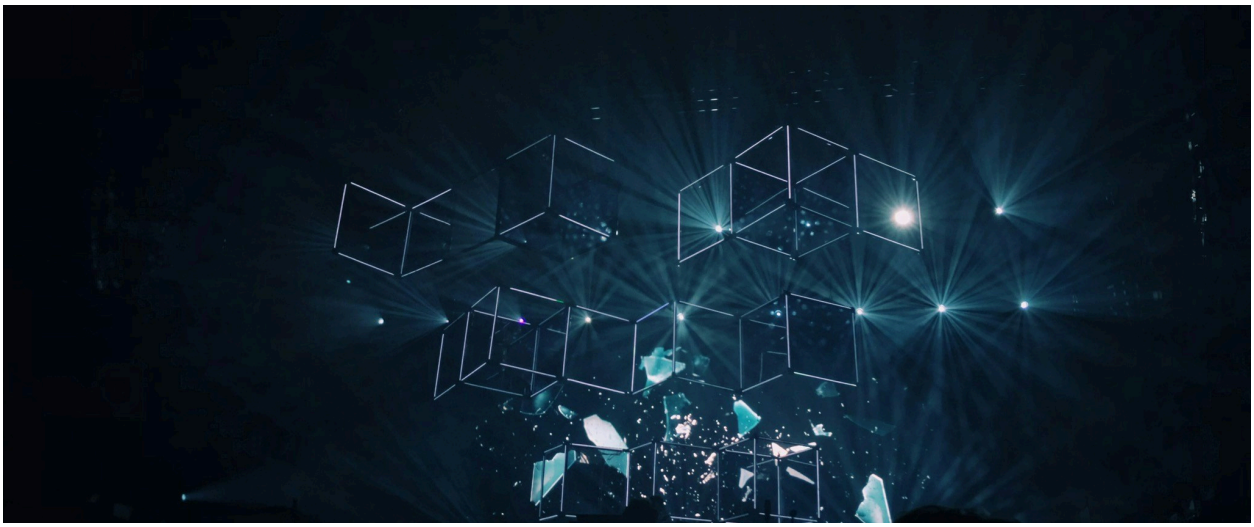

SQL Functions

By Kranti Patil
Fusion Software Institute

String Functions

LENGTH

Purpose: Returns the number of characters in a string

Syntax: `LENGTH(string)`

Example:

```
SELECT LENGTH('Hello World') AS string_length;
-- Result: 11

SELECT name, LENGTH(name) AS name_length
FROM employees
WHERE LENGTH(name) > 10;
```

CONCAT

Purpose: Combines two or more strings into one string

Syntax: `CONCAT(string1, string2, ...)`

Example:

```
SELECT CONCAT('Hello', ' ', 'World') AS full_greeting;
-- Result: Hello World

SELECT CONCAT(first_name, ' ', last_name) AS full_name
FROM employees;

SELECT CONCAT('Employee ID: ', employee_id, ' - Name: ', name) AS employ
ee_info
FROM employees;
```

SUBSTR (SUBSTRING)

Purpose: Extracts a substring from a string starting at a specified position

Syntax: `SUBSTR(string, start_position, length)`

Example:

```
SELECT SUBSTR('Hello World', 1, 5) AS substring_result;
-- Result: Hello

SELECT SUBSTR('Hello World', 7) AS substring_from_position;
-- Result: World

SELECT name, SUBSTR(name, 1, 3) AS name_prefix
FROM employees;

-- Extract year from date string
SELECT SUBSTR(hire_date, 1, 4) AS hire_year
FROM employees;
```

INSTR

Purpose: Returns the position of the first occurrence of a substring within a string

Syntax: `INSTR(string, substring)`

Example:

```
SELECT INSTR('Hello World', 'World') AS position;
-- Result: 7

SELECT INSTR('Hello World', 'xyz') AS not_found;
-- Result: 0 (not found)

SELECT name, INSTR(name, 'a') AS first_a_position
FROM employees
WHERE INSTR(name, 'a') > 0;
```

UPPER

Purpose: Converts all characters in a string to uppercase

Syntax: `UPPER(string)`

Example:

```
SELECT UPPER('hello world') AS uppercase_string;
-- Result: HELLO WORLD

SELECT name, UPPER(name) AS name_upper
FROM employees;

-- Case-insensitive search
SELECT * FROM employees
WHERE UPPER(name) = UPPER('john smith');
```

LOWER

Purpose: Converts all characters in a string to lowercase

Syntax: `LOWER(string)`

Example:

```
SELECT LOWER('HELLO WORLD') AS lowercase_string;
-- Result: hello world

SELECT name, LOWER(name) AS name_lower
FROM employees;

-- Email formatting
SELECT LOWER(email) AS formatted_email
FROM employees;
```

Window Functions

RANK()

Purpose: Assigns a rank to each row with gaps in ranking for tied values

Syntax: `RANK() OVER (ORDER BY column)`

Example:

```
-- Basic ranking
SELECT name, salary,
    RANK() OVER (ORDER BY salary DESC) AS salary_rank
FROM employees;

-- Ranking within departments
SELECT name, department, salary,
    RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS de
pt_rank
FROM employees;
```

ROW_NUMBER()

Purpose: Assigns a unique sequential number to each row

Syntax: `ROW_NUMBER() OVER (ORDER BY column)`

Example:

```
-- Sequential numbering
SELECT ROW_NUMBER() OVER (ORDER BY hire_date) AS row_num,
      name, hire_date
FROM employees;

-- Pagination example
SELECT * FROM (
   SELECT ROW_NUMBER() OVER (ORDER BY employee_id) AS rn,
         name, salary
   FROM employees
) WHERE rn BETWEEN 11 AND 20;  -- Page 2, 10 records per page
```

DENSE_RANK()

Purpose: Assigns ranks without gaps, even when there are tied values

Syntax: `DENSE_RANK() OVER (ORDER BY column)`

Example:

```
-- Dense ranking comparison
SELECT name, salary,
      RANK() OVER (ORDER BY salary DESC) AS regular_rank,
      DENSE_RANK() OVER (ORDER BY salary DESC) AS dense_rank
FROM employees;

-- Top 3 earners in each department (no gaps)
SELECT * FROM (
   SELECT name, department, salary,
         DENSE_RANK() OVER (PARTITION BY department ORDER BY salary DE
SC) AS rank
   FROM employees
) WHERE rank <= 3;
```

OVER Clause

Purpose: Defines the window or subset of rows for window functions

Syntax: `OVER (PARTITION BY column ORDER BY column)`

Key Components:

- **PARTITION BY:** Divides rows into groups
- **ORDER BY:** Defines the order within each partition
- **ROWS/RANGE:** Defines the window frame (optional)

Example:

```
-- Running total
SELECT name, salary,
       SUM(salary) OVER (ORDER BY hire_date ROWS UNBOUNDED PRECEDIN
G) AS running_total
FROM employees;

-- Moving average (3-row window)
SELECT name, salary,
       AVG(salary) OVER (ORDER BY hire_date ROWS 2 PRECEDING) AS moving
_avg
FROM employees;
```

PARTITION BY

Purpose: Divides the result set into partitions for window function calculations

Example:

```
-- Department-wise statistics
SELECT name, department, salary,
       AVG(salary) OVER (PARTITION BY department) AS dept_avg_salary,
       MAX(salary) OVER (PARTITION BY department) AS dept_max_salary,
       MIN(salary) OVER (PARTITION BY department) AS dept_min_salary
FROM employees;

-- Percentage of department total
```

```
SELECT name, department, salary,
       salary * 100.0 / SUM(salary) OVER (PARTITION BY department) AS pct_of
_dept_total
FROM employees;
```

Aggregate Functions

MAX

Purpose: Returns the maximum value in a set of values

Syntax: `MAX(column)`

Example:

```
-- Overall maximum
SELECT MAX(salary) AS highest_salary
FROM employees;

-- Maximum by group
SELECT department, MAX(salary) AS max_dept_salary
FROM employees
GROUP BY department;

-- Window function version
SELECT name, salary,
       MAX(salary) OVER () AS company_max,
       MAX(salary) OVER (PARTITION BY department) AS dept_max
FROM employees;
```

MIN

Purpose: Returns the minimum value in a set of values

Syntax: `MIN(column)`

Example:

```
-- Overall minimum
SELECT MIN(salary) AS lowest_salary
FROM employees;

-- Minimum by group
SELECT department, MIN(salary) AS min_dept_salary
FROM employees
GROUP BY department;

-- Finding employees with minimum salary in each department
SELECT name, department, salary
FROM employees e1
WHERE salary = (SELECT MIN(salary) FROM employees e2 WHERE e2.depart
ment = e1.department);
```

AVG

Purpose: Returns the average (arithmetic mean) of a set of values

Syntax: `AVG(column)`

Example:

```
-- Overall average
SELECT AVG(salary) AS average_salary
FROM employees;

-- Average by group
SELECT department, AVG(salary) AS avg_dept_salary
FROM employees
GROUP BY department;

-- Employees above department average
SELECT name, department, salary,
       AVG(salary) OVER (PARTITION BY department) AS dept_avg
FROM employees
```

```
WHERE salary > (SELECT AVG(salary) FROM employees e2 WHERE e2.depart
ment = employees.department);
```

SUM

Purpose: Returns the sum of all values in a set

Syntax: `SUM(column)`

Example:

```
-- Total payroll
SELECT SUM(salary) AS total_payroll
FROM employees;

-- Sum by department
SELECT department, SUM(salary) AS dept_total_salary
FROM employees
GROUP BY department;

-- Running sum
SELECT name, salary,
       SUM(salary) OVER (ORDER BY hire_date) AS running_total
FROM employees;

-- Percentage of total
SELECT name, salary,
       salary * 100.0 / SUM(salary) OVER () AS pct_of_total
FROM employees;
```

COUNT

Purpose: Returns the number of rows or non-NULL values

Syntax:

- `COUNT(*)` - counts all rows
- `COUNT(column)` - counts non-NULL values

- `COUNT(DISTINCT column)` - counts unique non-NULL values

Example:

```
-- Total employees
SELECT COUNT(*) AS total_employees
FROM employees;

-- Employees by department
SELECT department, COUNT(*) AS employee_count
FROM employees
GROUP BY department;

-- Count non-NULL values
SELECT COUNT(phone_number) AS employees_with_phone
FROM employees;

-- Count distinct values
SELECT COUNT(DISTINCT department) AS number_of_departments
FROM employees;

-- Window function version
SELECT name, department,
       COUNT(*) OVER (PARTITION BY department) AS dept_size
FROM employees;
```

Combining Multiple Functions

```
-- Complex string manipulation
SELECT
    name,
    UPPER(SUBSTR(name, 1, 1)) || LOWER(SUBSTR(name, 2)) AS proper_case,
    CONCAT(SUBSTR(name, 1, 3), '...') AS name_preview,
    LENGTH(name) AS name_length
FROM employees
```

```
WHERE INSTR(LOWER(name), 'a') > 0;

-- Advanced ranking with ties
SELECT
    name,
    department,
    salary,
    RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS salar
y_rank,
    DENSE_RANK() OVER (PARTITION BY department ORDER BY salary DESC)
AS dense_rank,
    ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC,
name) AS row_num
FROM employees;

-- Statistical analysis
SELECT
    department,
    COUNT(*) AS employee_count,
    MIN(salary) AS min_salary,
    MAX(salary) AS max_salary,
    AVG(salary) AS avg_salary,
    SUM(salary) AS total_salary,
    MAX(salary) - MIN(salary) AS salary_range
FROM employees
GROUP BY department
HAVING COUNT(*) > 5;
```

Performance Tips

> - Use `ROW_NUMBER()` instead of `RANK()` when you need unique sequential numbers

> - `DENSE_RANK()` is useful when you want consecutive ranks without gaps

> - Always include `ORDER BY` in window functions for consistent results

> - Use `PARTITION BY` to reset calculations for different groups

> - Consider indexing columns used in `ORDER BY` and `PARTITION BY` clauses